Case study of My Chat Project

Credits

Lead Developer: Alexander Eggleston
Tutor: Ezequiel De Simone
Mentor: Enrique Gonzalez

Overview

My unnamed chat app is a product of the second-to-last achievement of my CareerFoundry course, Full-Stack Immersion. The chat app itself is a “native app” also known as a phone application, it was coded using React Native, a framework for Javascript that allows for JS code to be converted into native code for phones. This is especially useful because iPhone and Android apps are coded in different languages, so a dedicated native app developer would need to know both of these to make each version of the same app.

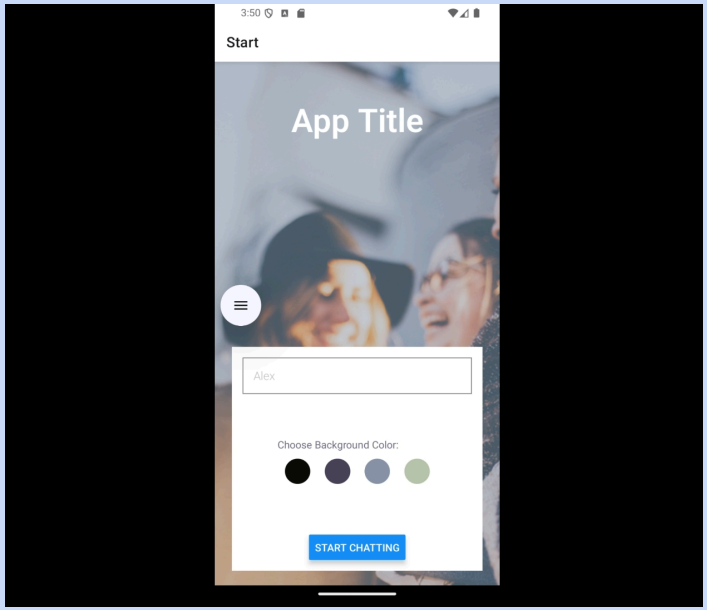
This app also gave me an insight into external data storage, through Firestore/Firebase, which is a Google-owned database host.

Purpose and Context

This app was a personal project, developed for my Full-Stack Immersion course, as mentioned above. Its purpose was to introduce me to native app development, user interfaces, accessibility, and external storage.

Objective

This project was minted to show me how to develop a native app, using React Native, Firestore, and Expo. The chat app exists to show that I can branch out into entirely new territory and still come out with a good addition to my portfolio.



A picture of the welcome page of the unnamed chat app.

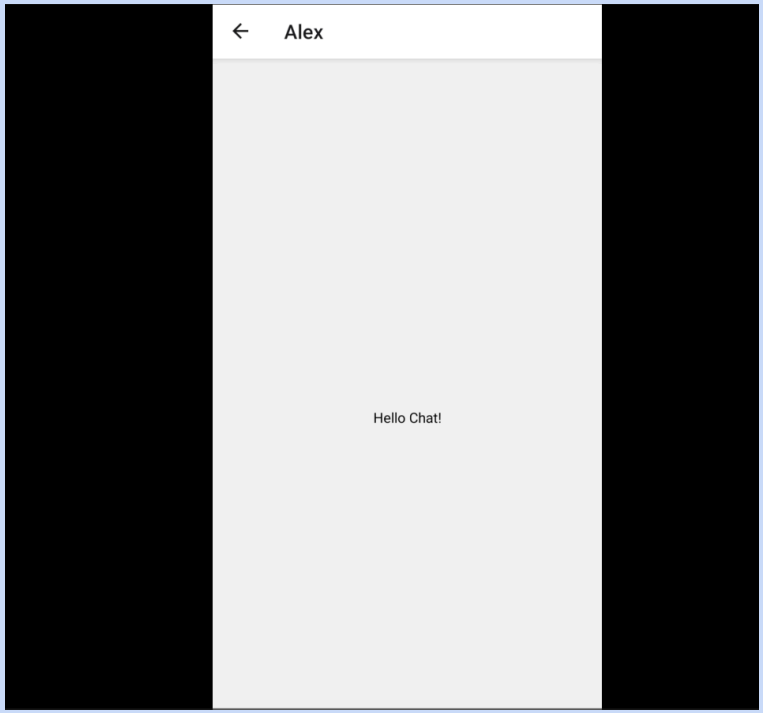
Duration

This project only lasted 5 deliverables, half the length of an average project, and it shows in the final product. That is not to say the final product is lacking in functionality, but it is very simple when compared to other projects I have worked on. It is just sending messages back and forth, and most of the deliverables were actually about other parts of the project, like the welcome page.

Development

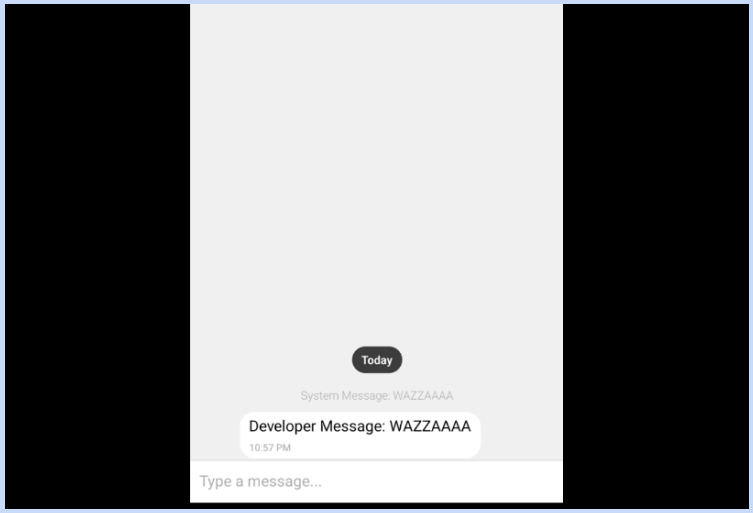
Since it was a native app in the midst of development, Chat wouldn't be in any app stores, but it also wouldn't run on my computer system, therefore development of the app began with the installation of an Android emulator. I also used Expo Go on my own digital phone to run Chat, but I ended up having issues interacting with the buttons, so I stuck with the Google Pixel 4 on my computer.

After this, I had to establish the framework of the app, which happened to be provided to me by CareerFoundry. You can see the welcome screen on the previous page. The chat page on the other hand…



The initial chat page… needed work.

This is closer to what we think of in a chat app, right?



The Chat Interface

I began by researching what other chat interfaces looked like, the contact pictures were circles, messages the user *sends* are on the right, messages they *receive* are on the left, there were additional buttons for recordings or attachments, and the top always gave information about *who* the user was chatting with. This is because that type of design is both user-friendly, and recognizable. Sometimes developers are encouraged to make something

unique to stick out from the pack, but in this case, it would be better for the user if the chat screen looked like something they recognize.

Accessibility

It is imperative that the internet and its services be given to everyone. Someone who is blind or deaf should receive as much knowledge and experience from the internet as I do, as someone who is neither, and to deny them opportunity is to rob them of one of the great shared experiences of humanity. For computer apps, the Web Accessibility Initiative has a set of guidelines and HTML tags that make said apps accessible. The issue arises in including these in native apps, as phones do not run on HTML. Luckily for me, both Apple and Android include system specific guidelines that can be included in React Native apps.

```
{/*Buttons to select the background color, each button, when selected, will send its color to the next screen*/}
{colorOptions.map((color, index) => (
  <TouchableOpacity
    accessibilityLabel={`Color: ${colorLabels[index]}`}
    key={`color-button__${color}`}
```

A snippet of the source code for the chat app, including an accessibility label that reads the color of a button.

External Storage

For this project, I also started up a free demo of Google's Firestore, which acts as server-side storage for all kinds of apps, sort of similar to the database I created myself in myFlix (Go read that case study too). Firestore allows the app to keep its data synced between users, and where does this come in handy? Messages! Every message sent in chat is kept in a server and distributed to each member of the chatroom so they know what you are saying.

Internal Storage

There is also a portion of data stored in the user's phone, such as the last few messages received, that way you can read them offline. It isn't much, but could you imagine going through a tunnel and all of your friend's messages disappearing?

Communication Features

The chat app also includes picture messaging, and location sharing. While pictures aren't anything new, they do use a different method than other messages, which took up a good portion of my learning. The location sharing, however, was a fun little addition that lets you send a small map attachment that shows your location to others, and links to Google Maps.

Challenges

The switch from React to React Native caused some adjustment issues, for example every mention of "click" is replaced with "press", which to me felt unnecessary. Other than things that were easy to overcome, I almost constantly had an issue with information not travelling between screens. You see dear reader, in native apps, each different screen layout exists alone, and needs to be given information from another screen, which I found constant trouble with.

For future developers, please triple check your capitalization, it will save you hours.

Reflection

My first thought when writing this case study is that I should have decided on a name. The working title for this project was just “Chat App”, which I jokingly changed to “The Chatler” later. That means… nothing really, but I didn’t have any ideas for a real title. This being such a late assignment, I had gone into it with some expectations, especially seeing how short it was going to be. It only took some three weeks to complete while I already had a job, and it didn’t leave much of an impression. To reflect on it more clearly, I am glad to have an insight into native app development, although mobile apps are, in my mind, a “full market” already, it would be challenging to come up with a truly groundbreaking native app in the current age.